

Exercise #3 - Fundamental Threshold Analysis, Segmentation and Tracking

Mason McGaffin

Exercise #3

ECEN 5763 Embedded Computer Vision

University of Colorado Boulder

June 26, 2026

Contents

1 THE WORLD OF INTERACTIVE MEDIA SYSTEMS AND APPLICATIONS.....	3
2 HOUGH LINES	4
2.1 INTERACTIVE HOUGH LINES.....	4
2.2 USE CASE.....	7
3 HOUGH CIRCLES.....	8
3.1 INTERACTIVE HOUGH CIRCLES.....	8
3.2 USE CASE	10
3.3 ADDING MORE INTERACTIVE FEATURES.....	10
4 BACKGROUND REMOVAL.....	13
5 ONE BAND GRAYSCALE GRAYMAP	15
6 CENTER OF MASS TRACKING.....	16
7 REFERENCES.....	20
8 APPENDIX	21

1 The world of interactive media systems and applications

Read the paper “The world of interactive media systems and applications” and summarize key points of the paper. At one point early in the days of ray tracing and computer vision, researchers worried about indistinguishable real video from generated (rendered) video – do you think this was a valid concern

The paper, written by Siewert, explores Linux development considerations for computer vision and interactive media systems. The paper first highlights the evolution of passive digital media to interactive media experiences. This transformation has changed the system architecture behind digital media systems, leveraging Cloud services as well. The author claims that systems no longer just transmit video, they must understand and interpret each frame.

Computer vision is a vital organ of interactive media applications. Siewert discusses the various applications of computer vision in everyday life that most people take for granted. From augmented reality to autonomous driving to facial recognition, computer vision methods are utilized under the hood. People do not realize the growth and impact that interactive digital media has had. At the close of the article, the author discusses the high potential of future interactive digital world.

The last key point of the paper is about Linux-specific development for computer vision. The article discusses tools, APIs, methods, and architectures suited to embedded computer vision applications. Siewert goes through examples and discusses 3D applications illustrating how computer vision can progress to solve complex tasks. The end of the article specifies that Linux-based computer vision expertise will be a highly valued skill given the current trends of computer vision.

Yes, the concern is valid. It is starting to become a very serious concern today. Especially with the development of artificial intelligence, it is becoming increasingly difficult to discern real video from generated video. I have seen firsthand the negative impacts this can have when used for harm. Ordinary people, especially those not familiar with the technology, become vulnerable to mass media manipulation if it is used maliciously. The early researchers had a valid concern, and it raises questions about whether stricter regulations should have and should still be in place regarding generated video.

2 Hough Lines

Starting with the OpenCV example for Hough Lines, adapt the code so you can compute Hough lines for a continuous camera stream (houghlines.cpp) and refer to class example for OpenCV 3.x as it may help you too, but you will have to update the code to get it to build and run (simple-hough-interactive/). Capture an image showing detection of lines for an object you hold up to the camera. What could you use this for?

2.1 Interactive Hough Lines

This application was created to allow the user to interactively explore Hough Line detection on both static images and live webcam video feed. The user can pass the image file as an argument to the execution call or pass the '-l' argument to use the live webcam mode. Then, using keyboard input, the user can switch between the original image/video ('R') and grayscale ('G') or apply standard Hough line detection ('S') and probabilistic Hough line detection ('P').

Initially, this application only provided the features listed above. Later in the development process, the application was combined with the work on Hough Circles to become one large interactive application, as described in [Section 3.3](#).

Hough line detection has the goal of determining borders. It calculates lines on each processed image/frame and attempts to identify distinguishable lines. As seen below, the probabilistic Hough line detection algorithm is much more accurate, at the cost of higher computing requirements. The examples below were captured using the default threshold, line length, and gap parameters provided in the OpenCV example.

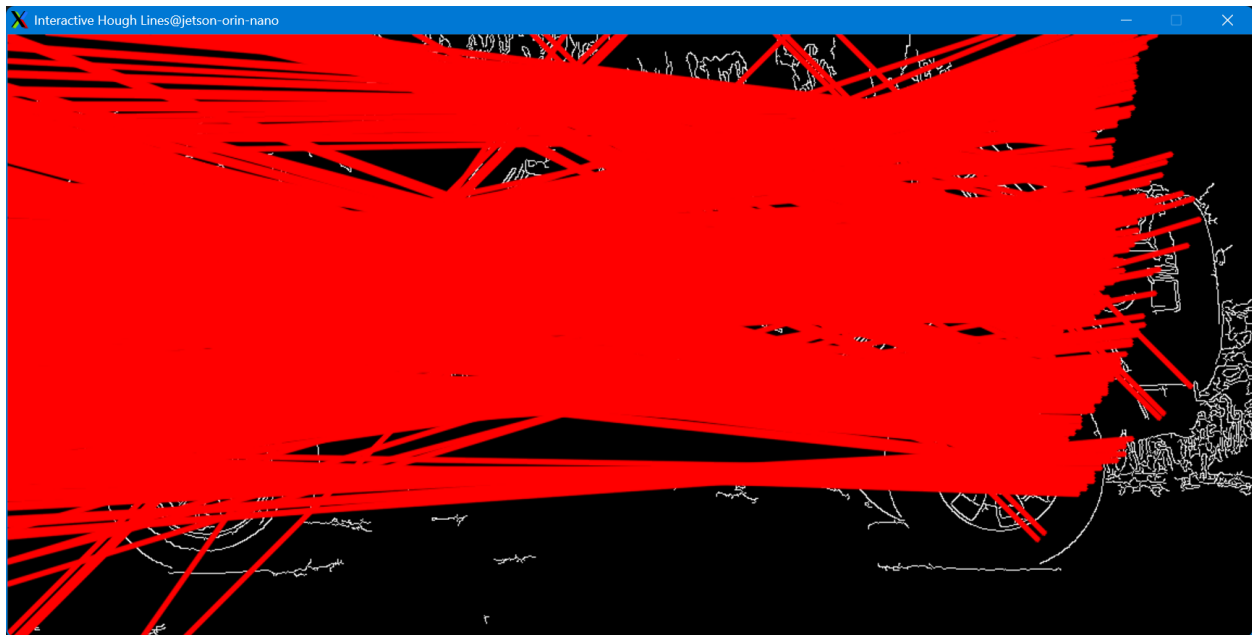


Figure 2.1 Standard Hough Detection on Jeep-Sideview.png



Figure 2.2 Probabilistic Hough line detection on Jeep-Sideview.png

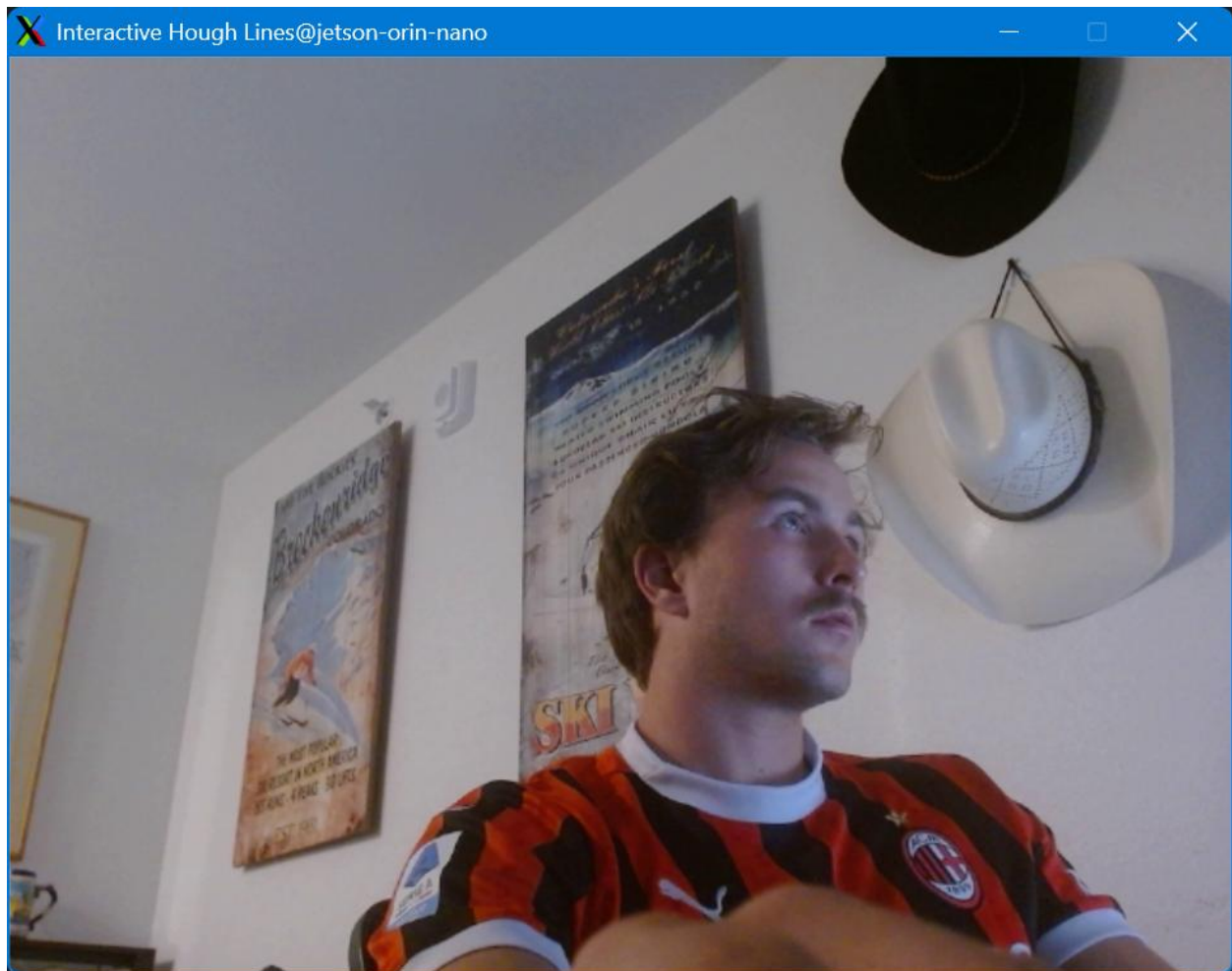


Figure 2.3 Unmodified live webcam capture

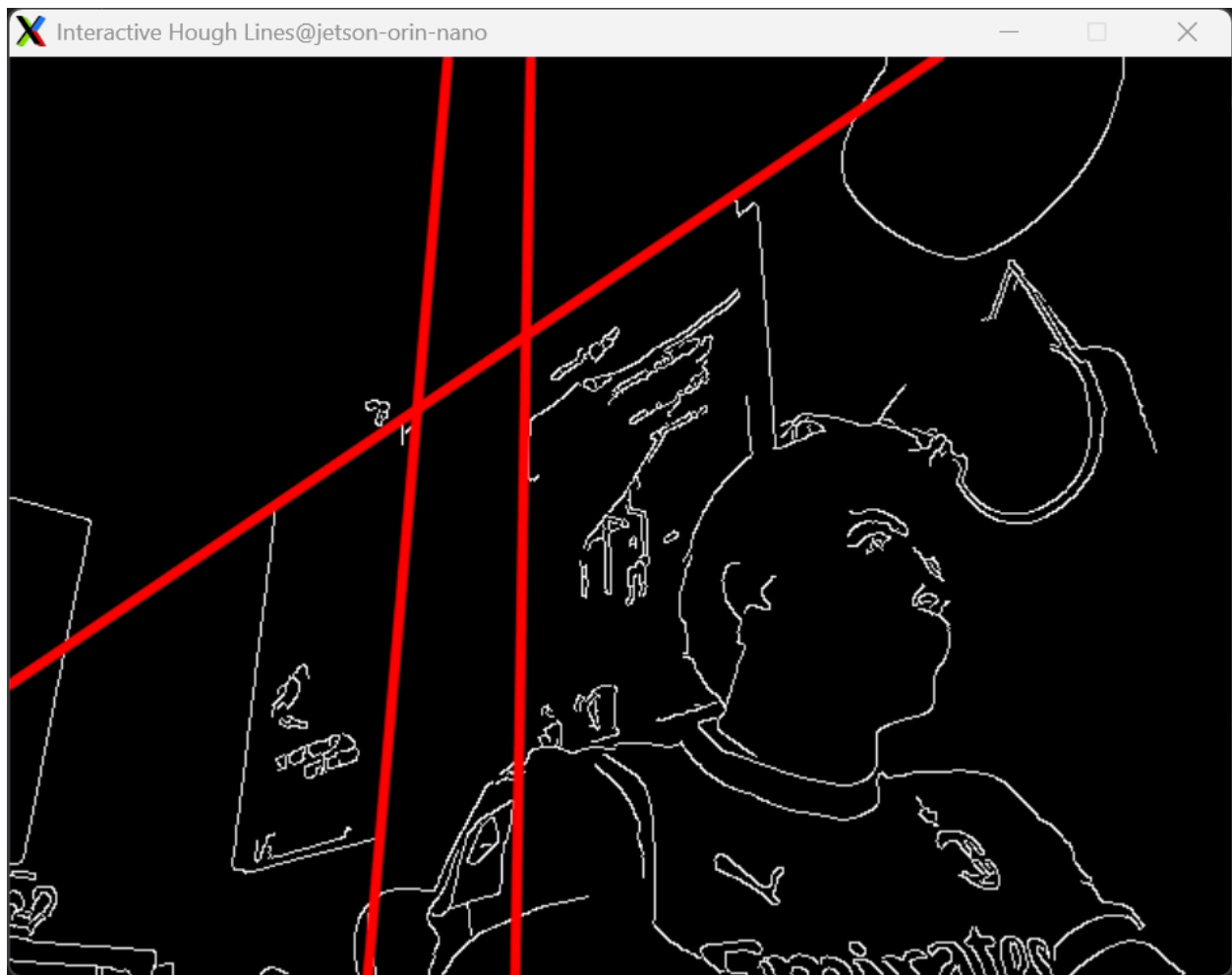


Figure 2.4 Standard Hough line detection on live webcam capture



Figure 2.5 Probabilistic Hough line detection on live webcam capture

As seen in the figures provided above, the probabilistic Hough line detection does a better job at identifying objects. The standard detection method is able to identify the boundaries but extends the lines, causing more difficult to understand output in this application.

2.2 Use case

Hough line detection is utilized heavily to detect straight lines and boundaries. Standard Hough line detection is used for applications that do not require line length or gap constraints. This includes combining dashed lines, extending lines, etc. Probabilistic Hough line detection aims at only tracking straight lines/edges for their entire length. Hough line detection is heavily utilized in autonomous navigation such as in robotics, automobiles, etc. It can also be used to identify, and/or correct alignment in industrial quality control and document checking. Hough line detection can also be used in pattern recognition. Specifically, it can be used in digitalizing games, such as chess, to identify the game board spaces and orientation.

3 Hough Circles

Starting with the OpenCV example for Hough Circles, adapt the code so you can compute Hough circles for a continuous camera stream (houghcircles.cpp) and refer to class example for OpenCV 3.x as it may help you too if you update it (simple-hough-ellipticalinteractive/). Capture an image showing detection of circles for an object you hold up to the camera. What could you use this for?

3.1 Interactive Hough Circles

This application allows the user to detect circles within a static image or a live webcam feed. As with the application for Hough line detection, the user can specify an input image or the live webcam mode. Initially the application used a static range for the minimum and maximum circle radius, as seen in Figure 3.1, Figure 3.2, and Figure 3.3. Section 3.3 discusses the additional interactive features that allow the user to combine Hough lines and circles, as well as trackbars for live dynamic interaction.

Figure 3.1 shows the live webcam capture of the Tau Beta Pi Engineering Honors Fraternity bent. The application was compiled with a radius range of one (1) to thirty (30) pixels. As seen in the figure, the circular top of the bent is drawn over by a pink circle. This circle indicates the Hough circle algorithm detected a probable circular object with radius within the range.

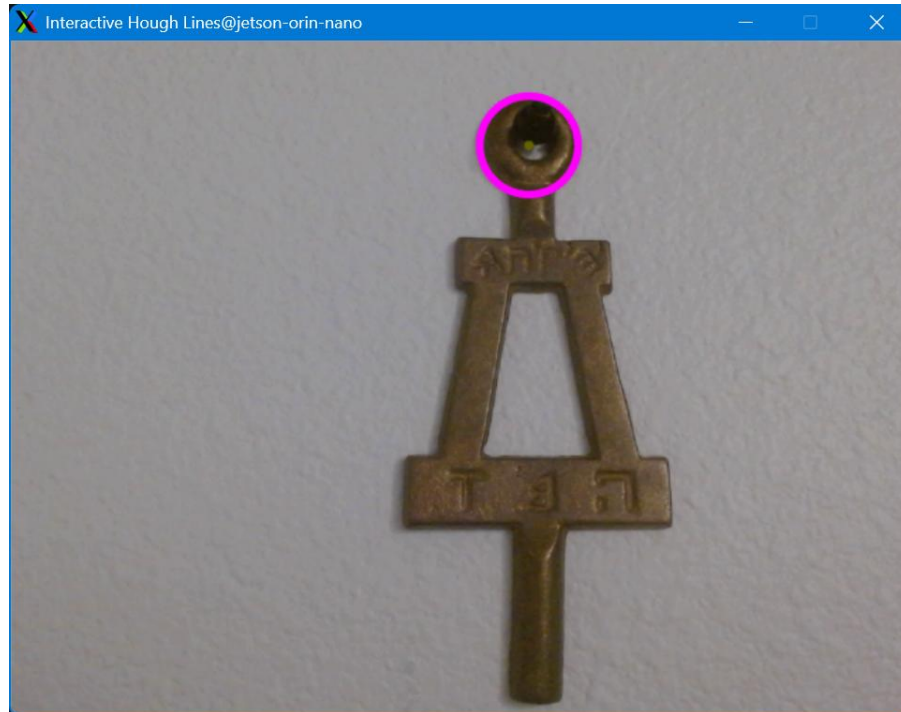


Figure 3.1 Hough circle detection using radius range 1-30

Figure 3.2 shows a live webcam capture of me holding up a large coin/ball marker. This time, the radius range was modified to be forty (40) to seventy (70) pixels. As seen below, the coin and the inner rim, near the hat band, of my cowboy hat are detected to be circular. By shifting the accepted radius range, the application is able to detect the ball marker and the hat while ignoring smaller circular objects such as parts of my knuckle and eyeball.

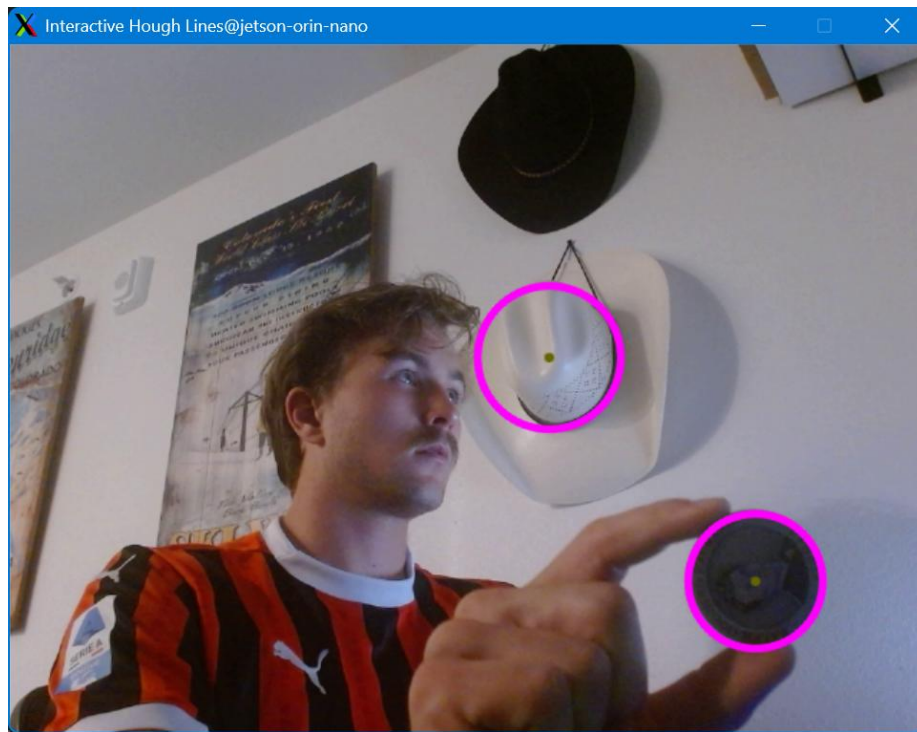


Figure 3.2 Hough circle detection using radius range 40-70

Figure 3.3 demonstrates the application use on a static PNG image. The radius range is configured as sixty (60) to seventy (70) pixels. This range is perfect to highlight the rims of the Jeep only.



Figure 3.3 Hough circle detection using radius range 60-70

3.2 Use Case

Hough circle detection has many of the same application of Hough line detection, just applied to a different shape. For industrial quality control, the method can be used in the beverage industry for can and bottle shapes. Autonomous navigation may use it to identify traffic signs or positional anchors. Hough circle detection is also heavily used in medical imaging such as X-ray. The method could also be used for eye-tracking applications such as gaze-controlled computers and medical/physiological testing.

3.3 Adding more interactive features

As mentioned before, the Hough line and Hough circle applications were combined into one larger interactive application. I added trackbars so the user could dynamically modify the threshold, length, and gap for Hough Line detection and modify the minimum and maximum radius for Hough Circle detection. Everything updates dynamically at runtime and there are guards against invalid inputs that break the application. Figure 3.4 shows the trackbars being utilized in Hough circle detection to isolate the wheels of the Jeep. Figure 3.5 and Figure 3.6 show the trackbars used to modify the output of the live webcam capture using standard and probabilistic Hough line detection, respectively.

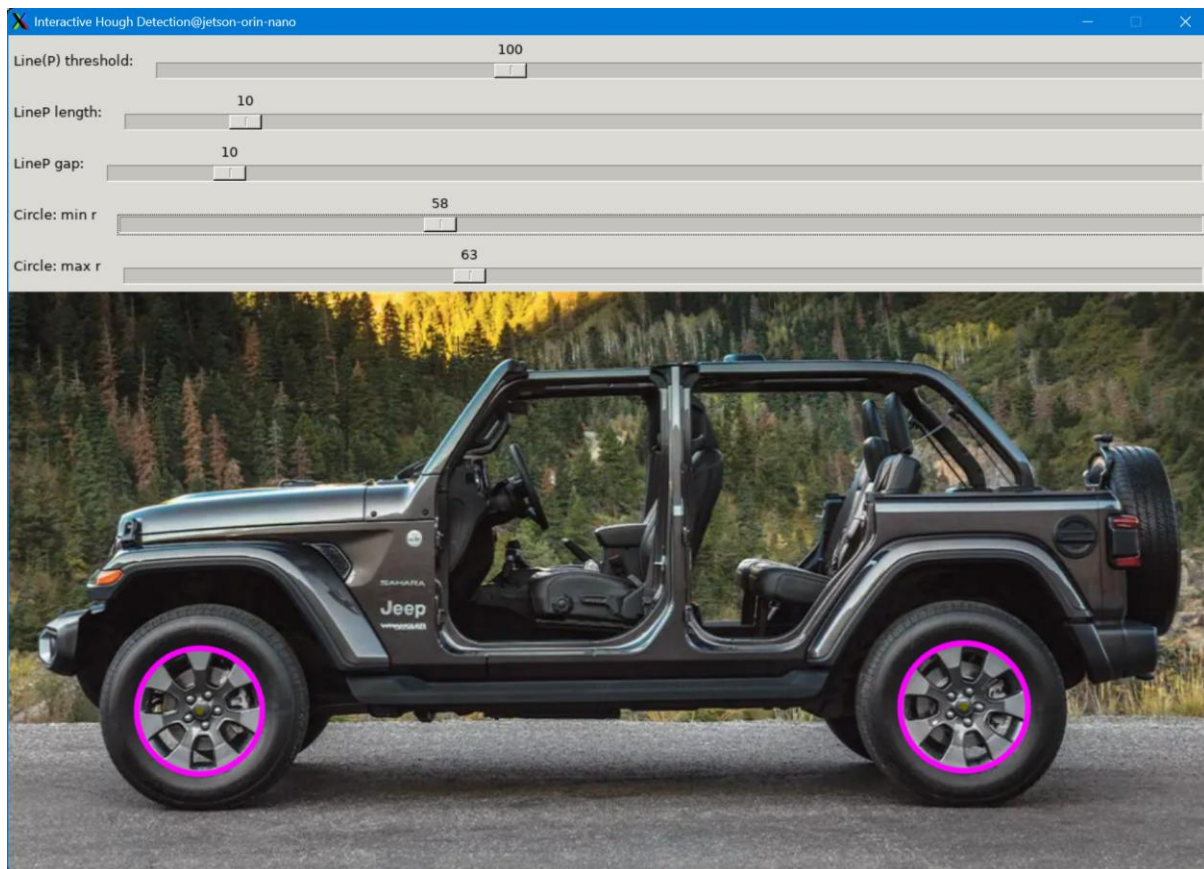


Figure 3.4 Interactive Hough circle detection on Jeep-Sideview.png

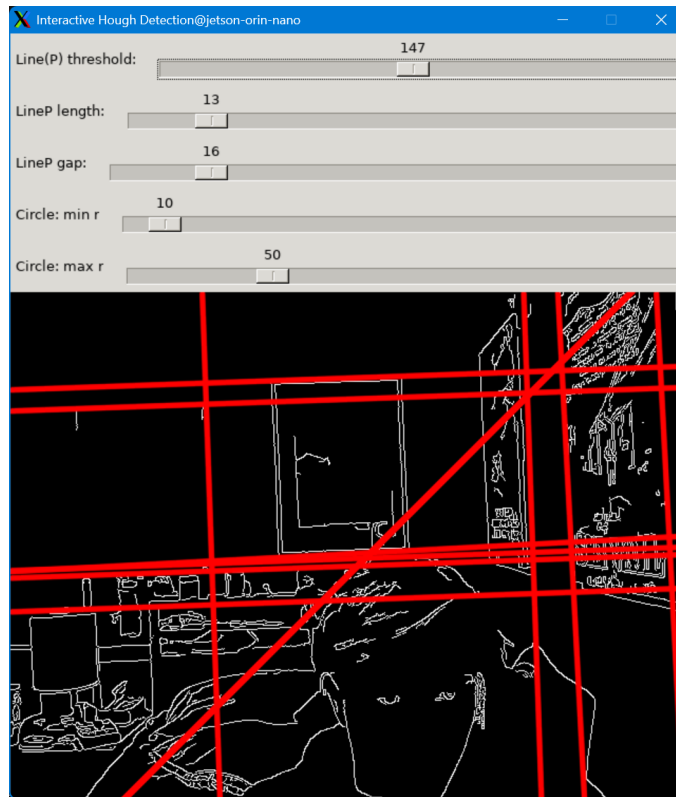


Figure 3.5 Interactive standard Hough line detection on live webcam capture

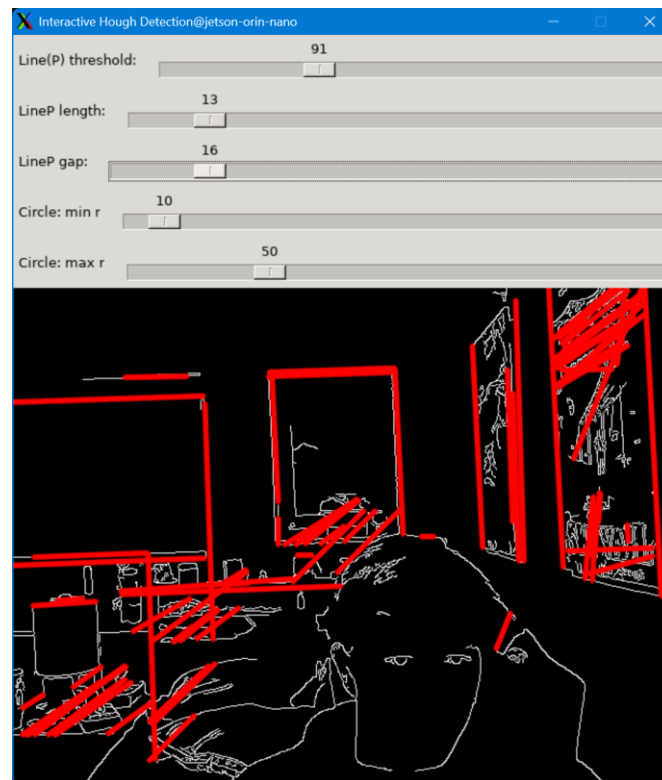


Figure 3.6 Interactive probabilistic Hough line detection on live webcam capture

Lastly, I added syslog outputs to log the open and close of the application, as well as the key press events that change the viewing mode, as seen in Figure 3.7.

```
Jun 25 09:03:19 jetson-orin-nano interactive_hough[4078093]: Interactive Hough Detection started using live webcam.
Jun 25 09:03:42 jetson-orin-nano interactive_hough[4078093]: Key pressed...Hough Standard Line Detection
Jun 25 09:03:45 jetson-orin-nano interactive_hough[4078093]: Key pressed...Hough Probabistic Line Detection
Jun 25 09:03:49 jetson-orin-nano interactive_hough[4078093]: Key pressed...Hough Circle Detection
Jun 25 09:03:50 jetson-orin-nano interactive_hough[4078093]: Key pressed...Grayscale Mode
Jun 25 09:03:53 jetson-orin-nano interactive_hough[4078093]: Key pressed...Original Image
Jun 25 09:03:58 jetson-orin-nano interactive_hough[4078093]: Closing application...
```

Figure 3.7 interactive-hough syslog output

4 Background removal

Process the video, Dark-Room-Laser-Spot-with-Clutter.mpeg and use color frame differencing for R,G & B to remove the bookshelf background and to preserve the moving laser spot foreground. How effective is this for removing background clutter? Show an example of the original video display frames with imshow and another with the difference image background removal using imshow for the same frame side by side (looks like - code/diff-interactive/tick-detection.png). You may find this background removal example for OpenCV 3.x useful and interesting (code/diff-interactive/).

This example relies on the usage of the `absdiff()` API. For each frame of the video, processed by the OpenCV VideoCapture class, the `absdiff()` function reports the difference in each pixel R,G,B value. This function is used instead of the subtraction operator in order to preserve negative differences between frames. The user can specify the input video file, output video file name (optional), or if they want to use a live interactive version. If the '-l' argument is not specified in the execution command, the application runs without the GUI and instead outputs a MPEG-4 (.mp4) video. The captured videos can be found in video.zip. The code can be easily modified, as shown in the following examples, to instead output PPM frames that can be used to encode video with the background removed in the original MPEG format. This method is extremely good at eliminating background clutter and only focuses on the foreground object (laser point). As seen in all three example below, the laser is easily identified. However, it is limited if the object is not moving as the frame difference will be less. Also, in the light room example, occasionally, a reflection of natural light and/or the laser would cause additional frame differences that added noise to the background removal.

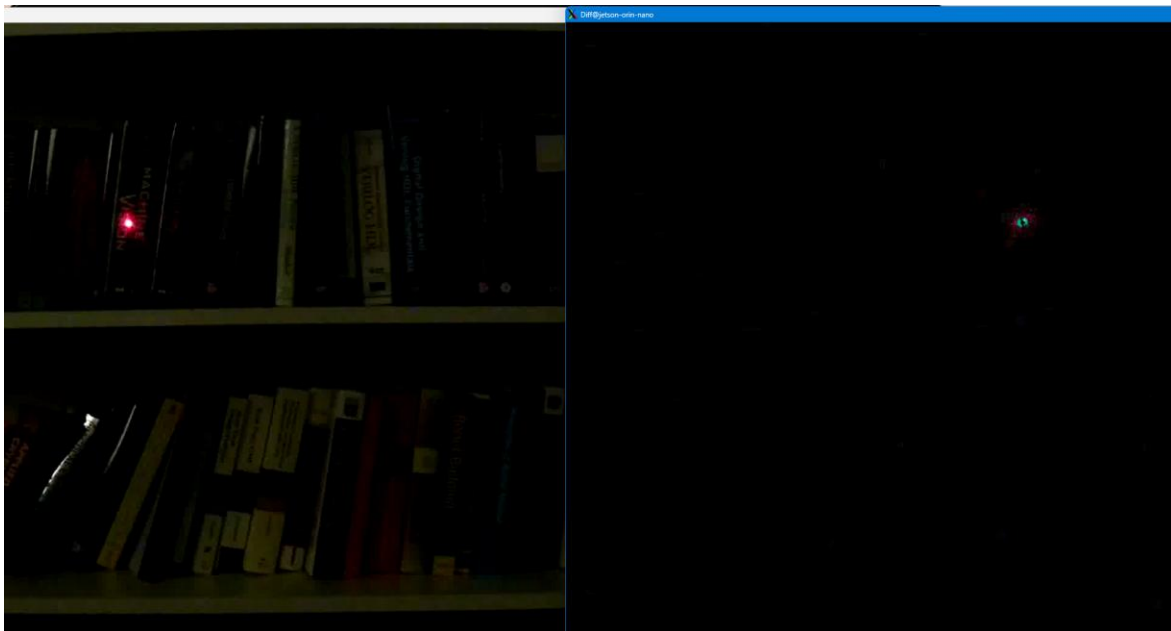


Figure 4.1 Dark-Room-Laser-Spot-with-Clutter.mpeg frame differencing

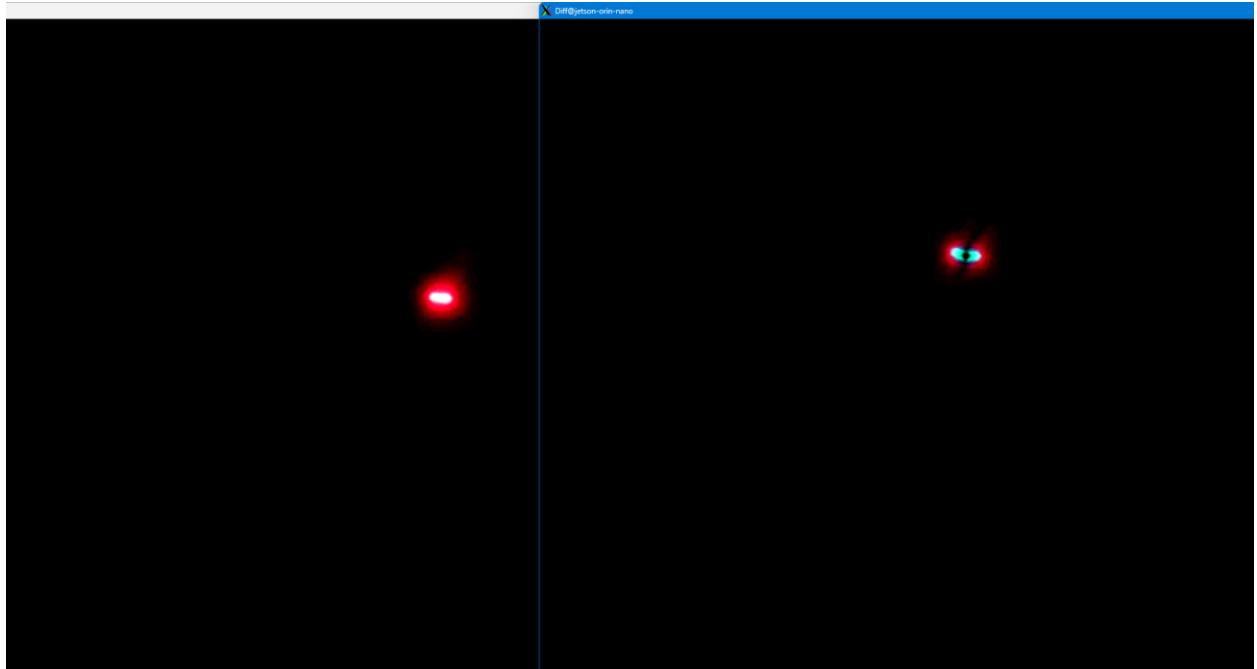


Figure 4.2 Dark-Room-Laser-Spot.mpeg frame differencing

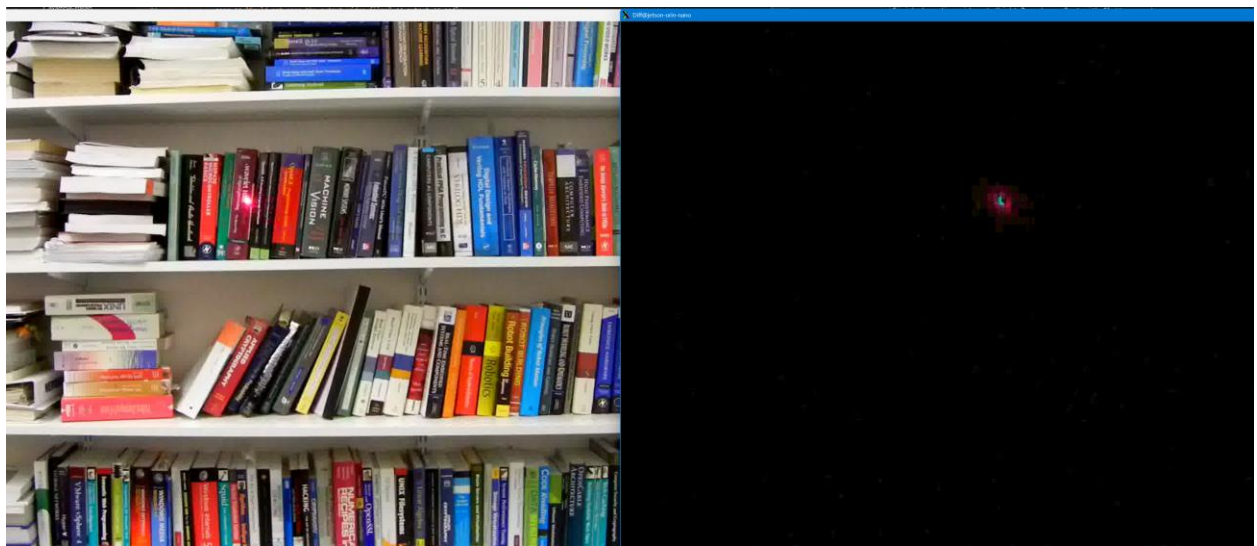


Figure 4.3 Light-Room-Laser-Spot-with-Clutter.mpeg frame differencing

5 One Band Grayscale Graymap

Use the Dark Room Laser Spot video (Dark-Room-Laser-Spot.mpeg) and convert the entire video to a grayscale graymap by using ONE band (from RGB) only - re-encode the video. You can do this using OpenCV and/or a combination of OpenCV and ffmpeg. Note that frames can be saved as a PGM (see http://en.wikipedia.org/wiki/Netpbm_format) and then re-encoded into MPEG4 using ffmpeg. You can use OpenCV if you wish or your own code to write out the PGM frames in a format suitable for ffmpeg encoding. Note that ffmpeg needs filenames to be of the format name%04d.pgm or similar so that frames are encoded in order. Consider more cluttered and lower contrast laser tracker video for extra challenge if you wish (Dark-Room-Laser-Spot-with-Clutter.mpeg, Light-Room-Laser-Spotwith-Clutter.mpeg).

For this exercise, we use another approach to attempt to isolate a foreground object. The example relies on the OpenCV `split()` API. This function separates each frame by the three color channels, B, G, R, into separate single dimension images, as Mat objects of type `CV_8U1`.

The first application, `cv_oneband_frames`, takes user input for the input file and color band. Then, the application writes each single dimension grayscale graymap, using the band specified, as a PGM file. After each frame is written as a single band graymap (PGM), `ffmpeg` is used to re-encode the video. The frames can be found in `frames.zip` and the generated video is in `video.zip`.

```
ffmpeg -r 30 -i frames/gray_<FILE>_%06d.pgm frames_<FILE>%06d.mp4
```

Command to re-encode grayscale graymap frames as MPEG video

The second application, `cv_oneband_mp4`, is very similar to `cv_oneband_frames`, except the final video is created using the OpenCV `VideoWriter` class. It would take very limited effort to combine the two applications and rely on arguments to specify which function to execute.

As mentioned above, both of these application complete the same function – applying a single band grayscale graymap to a video. This method, with regard to object tracking, creates a much more reliable output of the foreground object, but it has more trouble with background clutter. In order to mitigate this, we combine the grayscale graymap with the frame differencing from [Section 4](#). This creates a very clear detection of the laser as it moves across the screen. The video can be found in `video.zip`.

6 Center of Mass Tracking

Use the Dark Room Laser Spot video you converted to a series of graymap frames (using ONE band from RGB) and determine a threshold function based on your analysis of the characteristics of the edges of this spot in ONE band (using GIMP or MATLAB may help you). Raster each frame to determine the X-bar, Y-bar object COM (Center of Mass) based on X row maximum extents and Y column maximum extents and your threshold detector. Use any edge detector or convolution you want to enhance the image prior to threshold detection of the X and Y edges, or just use the raw data and a well designed and determined threshold. Mark the COM (Center of Mass) and track it in each frame with crosshairs (at saturation level of 255 - should appear as white lines). Save your annotated video showing tracking as an MPEG-4 (either using OpenCV or by saving frames and then encoding the frames using ffmpeg).

For this application, a single-band grayscale gray map is applied to isolate the red color of the laser. Then, frame differencing is used to eliminate background clutter. This modified image ideally shows only the target laser object. However, clutter, reflected light, laser reflection, etc. cause additional unwanted pixels to have positive value. Therefore, a threshold function is applied to eliminate noise. Once the object has been isolated, the application predicts the object location on the frame using a center of mass function, Code 6.4. This function calculates the row and column maximum extents for each frame to generate a predicted central point of the laser point. This location is used to draw a crosshair centered at the calculated location of the laser point object. The crosshair is drawn as perpendicular lines of length 200 pixels intersecting at the midpoint which is the center of mass calculated prior. This demonstrates accurate object tracking, as seen in Figure 6.1, Figure 6.2, and Figure 6.3.

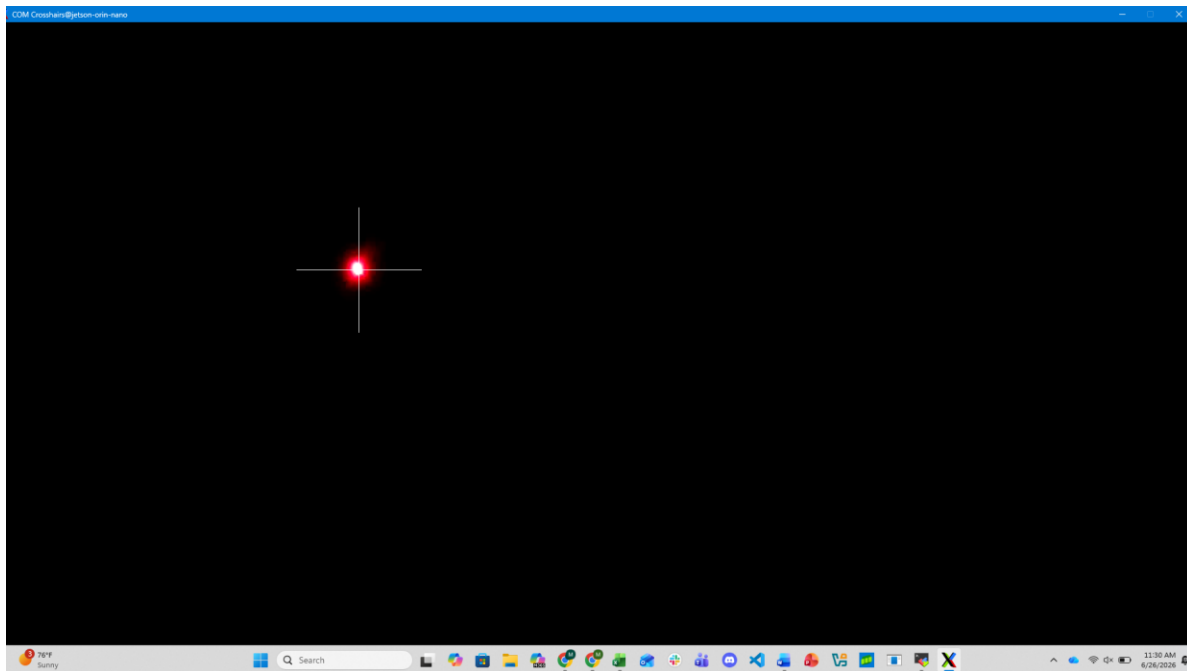


Figure 6.1 Dark-Room-Laser-Spot.mpeg center of mass tracking

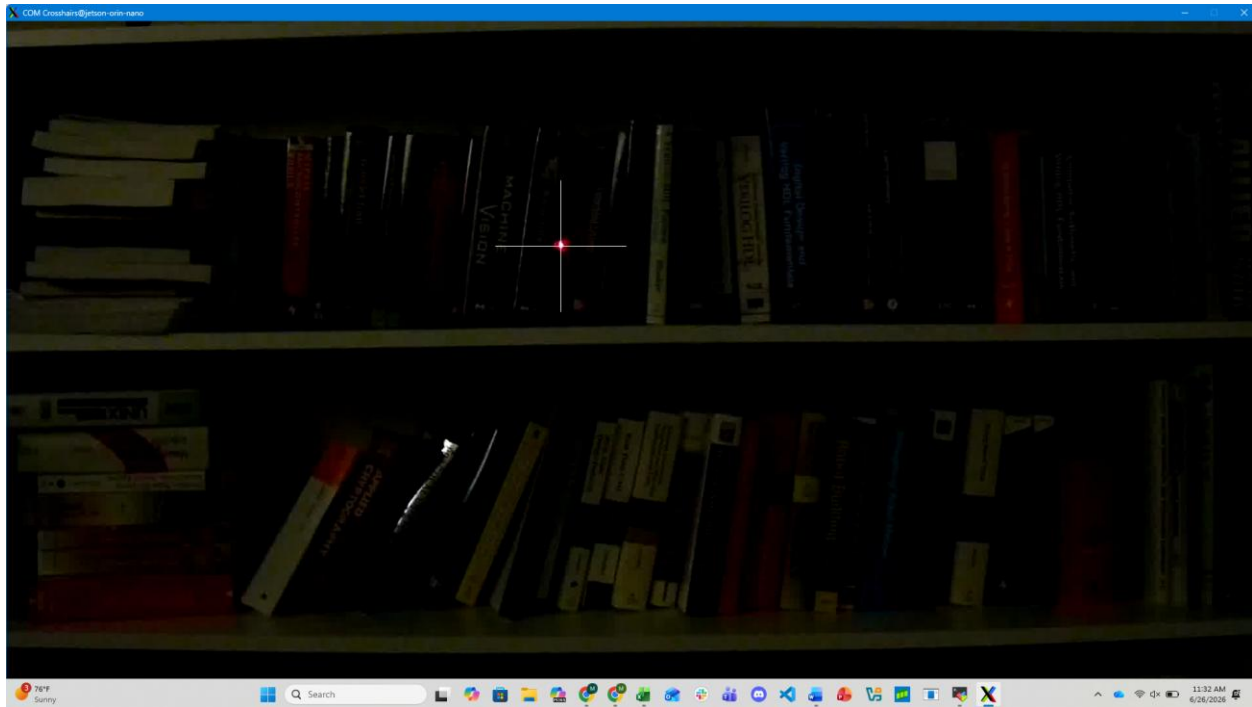


Figure 6.2 Dark-Room-Laser-Spot-with-Clutter.mpeg center of mass tracking



Figure 6.3 Light-Room-Laser-Spot-with-Clutter.mpeg center of mass tracking

One challenge observed was that the center of mass function struggles when the object stops moving. This is because of the frame differencing step of the application. To handle this, if the center of mass function is unable to calculate a reasonable position, the previous valid center of

mass is used. The reasoning is that if the object is not moving, then its center of mass is not moving either. Another difficulty faced, primarily with the Light-Room-Laser-Spot-with-Clutter.mpeg video, was the reflecting light of the laser. Some of these reflections were very bright, causing the originally calculated threshold to be too low. Most of these issues were resolved by increasing the threshold but it did not fix all of the mistracking. As future work, a method to calculate momentum of the center of mass could be used to check sharp changes. However, this would need to be fine tuned according to the frame rate and the speed of the object moving as sharp, quick direction changes would cause less accurate tracking.

```
void getCOM(COM *com, Mat frame)
{
    int xMin = INT_MAX;
    int xMax = -1;
    int yMin = INT_MAX;
    int yMax = -1;

    for(int r=0; r < frame.rows; r++)
    {
        for(int c=0; c < frame.cols; c++)
        {
            unsigned char val = frame.at<unsigned char>(r, c);
            if(THRESHOLD < val)
            {
                xMin = min(xMin, c);
                xMax = max(xMax, c);
                yMin = min(yMin, r);
                yMax = max(yMax, r);
            }
        }
    }

    if(xMax == -1 || xMin == INT_MAX) // pick any - if original state -
        invalid
        return;

    com->x = (xMin+xMax)/2;
    com->y = (yMin+yMax)/2;
}
```

Code 6.4 getCOM() function

As mentioned above, a threshold was used to remove unwanted clutter, reflections, etc from the frame. In order to calculate the threshold, ImageMagick was used to analyze the spot color intensity dropoff using the features to convert the frame data to histograms and text. The tool was used to plot the spot difference drop-off. For this exercise, this was done statically, outside the execution of the com_crosshair application, and the threshold value was updated manually by the user. It should be noted that the threshold had to be adjusted based on the input video. In future

work, this could be implemented dynamically, using the pixel values of each PGM frame to dynamically calculate a threshold.

7 References

- [1] FFmpeg Developers, "FFmpeg Frequently Asked Questions," *FFmpeg Official Documentation*, 2026. [Online]. Available: <https://ffmpeg.org/faq.html>
- [2] OpenCV, "OpenCV 4.8.0 Documentation." [Online]. Available: <https://docs.opencv.org/4.8.0/index.html>.
- [3] Siewert, IBM Corporation, "Interactive Video System," *IBM Out-of-Print Technical Documentation*, Shared Course Resources via University of Colorado Boulder. [Online]. Available: https://o365coloradoedu-my.sharepoint.com/:b:/r/personal/siewerts_colorado_edu/Documents/ESEE-Web-Resources/ECV-5763-CU/Documents/Papers/IBM-out-of-print/bd-interactive-pdf.pdf
- [4] "Netpbm format," *Wikipedia, The Free Encyclopedia*, Jun. 2026. [Online]. Available: https://en.wikipedia.org/wiki/Netpbm_format

8 Appendix

Github repository: https://github.com/mamc3334/ECV_5763/ex3

Pull request: https://github.com/mamc3334/ECV_5763/pull/4

Forked ECV Github repository: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin>

Pull request: N/A